Ders 22 Çalışma Özeti

Ders 22 Çalışma Özeti

Genel Konular

- Alt dizgi arama (substring search) problemi: uzun bir metinde (text) kısa bir örüntüyü (pattern) bulmak.
- Kaba kuvvet (brute force) arama: algoritma, C kodu ve karmaşıklık analizi.
- Boyer-Moore yöntemi: örüntüyü sondan başa karşılaştırma ve uyumsuz karaktere göre kaydırma.
- Kaydırma tablosu (`rightSkip`, Horspool yaklaşımı) ile kaydırma miktarının önceden hesaplanması.
- Rabin-Karp parmak izi (fingerprint) yöntemi: modüler özetleme (modular hashing) ve kayan özet (rolling hash).
- Üç yöntemin karmaşıklık karşılaştırması.

Hocanın Özellikle Vurguladığı Kısımlar

- Arama ve sıralama, en sık ihtiyaç duyulan algoritmalardır. Bu ders tek eleman yerine bir grup bilgiyi (DNA dizisi içinde parça, metin içinde kelime) aramayı ele alıyor.
- Kaba kuvvet aramada her uyumsuzlukta örüntü bir karakter kaydırılır ve karşılaştırma örüntünün başından yeniden başlar. Metindeki karakterlere defalarca bakılır.
- Arama `i <= n - m` noktasına kadar sürer. Geriye örüntüden kısa bir parça kaldığında aramaya gerek yoktur.
- Fonksiyon içinden `return` ile çıkmak, döngüyü `break` ile kırmakla aynı şey değildir. Hoca `break` ile döngü kırmamayı, okunabilirlik için `while` ve `found` bayrağı kullanmayı önerdi.
- Boyer-Moore'da karşılaştırma örüntünün sonundan başına doğru yapılır.
- Uyumsuz karakter örüntüde yoksa örüntü o karakterin tamamen ötesine kaydırılır. Varsa örüntüdeki en sağdaki (rightmost) geçişi o karakterle hizalanır.
- Örüntü hiç değişmediği için kaydırma miktarları önceden bir tabloya yazılabilir. Algoritmanın asıl güzelliği budur: her uyumsuzlukta örüntüyü taramak yerine tablodan bakılır.
- Hesaplanan kaydırma negatif çıkarsa geri gidilmez, `i` bir artırılır. Geri gidilirse daha önce bakılmış yerlere dönülür.
- Rabin-Karp'ta özetler eşit çıkarsa yine karakter karakter karşılaştırma yapılmalıdır. Farklı sayıların modu aynı çıkabilir (çakışma, collision).
- Mod için büyük bir asal sayı seçilir. Bu, çakışma ihtimalini azaltır ama sıfırlamaz.
- Her pencerenin sayısını baştan hesaplamak yerine bir önceki pencerenin değerinden yararlanmak çok daha verimlidir.

Kısa Tekrar Notları

- N: metin uzunluğu, M: örüntü uzunluğu.
- Kaba kuvvet: en iyi durum yaklaşık N, en kötü durum $(N - M + 1) \cdot M \approx NM$ karşılaştırma.
- Kaba kuvvetin en kötü durumu: metin `AAA...AB`, örüntü `AAAB` gibi sürekli tekrar eden karakterler.
- Boyer-Moore kaydırma kuralı: `skip = j - rightSkip[t[i+j]];` `skip < 1` ise `skip = 1;` sonra `i += skip`.
- `rightSkip` tablosu: örüntüdeki her karakter için en sağdaki indisi, örüntüde olmayan karakterler için -1 tutar.

- Boyer-Moore: tipik durumda yaklaşık N/M karşılaştırma, en kötü durumda NM.
- Rabin-Karp: örüntünün özeti ile metindeki her M uzunluklu pencerenin özeti karşılaştırılır. Pencere birer birer kayar.
- Kayan özet: $x_{i+1} = (x_i - t_i \cdot B^{M-1}) \cdot B + t_{i+M}$, hepsi mod$Q$.

Detaylı Açıklamalar

Kaba Kuvvet Arama

Örüntü metnin başına oturtulur. Karakterler soldan sağa karşılaştırılır. Uyumsuzluk çıkarsa örüntü bir sağa kaydırılır ve karşılaştırma örüntünün başından yeniden başlar.

- **En iyi durum.** Örüntünün ilk harfi metinde neredeyse hiç geçmiyorsa (örneğin `NEEDLE` aranırken metinde başka `N` yoksa) her konumda tek karşılaştırma yapılır. Toplam yaklaşık N karşılaştırma.
- **En kötü durum.** Metin `AAAAAAAAAB`, örüntü `AAAB` olsun. Her konumda ilk üç `A` uyar, `B` uymaz. Her konumda M karşılaştırma yapılır. Son bakılabilecek konum $N - M$ olduğu için toplam $(N - M + 1) \cdot M$ karşılaştırma.
- M küçükse (100, 200 gibi) bu yaklaşık $O(N)$ sayılır. M büyüdükçe NM çarpımı $O(N^2)$'ye yaklaşır.

Hocanın ilk yazdığı kod (`for` ve `return` ile):

```
int search(char p[], char t[])
{
    int i, j, n, m;
    n = strlen(t);
    m = strlen(p);
    for (i = 0; i <= n - m; i++) {
        j = 0;
        while ((j < m) && (p[j] == t[i + j]))
            j++;
        if (j == m)
            return i;       /* bulundu: örüntünün metindeki başlangıcı */
    }
    return -1;              /* bulunamadı */
}
```

Bir öğrencinin `return`'ün döngüyü kırıp kırmadığını sorması üzerine hoca aynı algoritmayı `while` ve `found` bayrağıyla yeniden yazdı:

```
i = 0; found = 0;
while ((i <= n - m) && (!found)) {
    j = 0;
    while ((j < m) && (p[j] == t[i + j]))
        j++;
    if (j == m)
        found = 1;
    else
        i++;
}
if (found == 1)
    return i;
else
    return -1;
```

- `j == m` ise örüntünün bütün karakterleri uymuştur.
- Döngüden çıkış koşulu `while` başlığında açıkça göründüğü için büyük programlarda okunması daha kolaydır.

Boyer-Moore Yöntemi

Örüntü metne oturtulur ama karşılaştırma örüntünün son karakterinden başlar. `j` değeri $M-1$'den 0'a doğru azalır.

Uyumsuzlukta üç durum vardır (tahtadaki hali):

Durum	Koşul	Yeni `i`
Case 1	Uyumsuz karakter örüntüde yok	`i = i + j + 1`
Case 2a	Karakter örüntüde var, kaydırma pozitif	`i = i + j - rightmost`
Case 2b	Karakter örüntüde var, kaydırma negatif	`i = i + 1`

- **Case 1 örneği.** `NEEDLE` aranırken metindeki `S` ile `E` uymaz. `S` örüntüde olmadığı için örüntü `S`'nin tamamen sağına kaydırılır.
- **Case 2a örneği.** Metinde `...SOKARA...`, örüntü `KARA` olsun ($i = 9$). Sondan karşılaştırınca son karakterler (`A` ile `A`) uyar. $j = 2$'de metindeki `K` ile örüntüdeki `R` uymaz. `K` örüntüde 0. indiste. Yeni $i = 9 + 2 - 0 = 11$. Böylece iki `K` hizalanır.
- **Case 2b örneği.** Örüntü `NEEDLE`, $i = 10$, $j = 3$'te uyumsuzluk olsun. Uyumsuz karakter `E` ise `E`'nin en sağdaki yeri 5'tir. $3 - 5 = -2$ çıkar; örüntü geri kayacaktır. Geri gitmek anlamsız olduğu için i yalnız bir artırılır.

Kaydırma Tablosu (`rightSkip`)

Örüntü değişmediği için her uyumsuzlukta "bu karakter örüntüde var mı, nerede?" diye aramaya gerek yoktur. Hoca bu fikri Nigel Horspool'a (1980'ler) bağladı.

- Alfabe yalnız 26 büyük harfse tablo boyutu 26'dır. `offset = 'A'` alınır: `'B' - 'A' = 1`, `'A' - 'A' = 0`.
- Küçük harfler için `offset = 'a'` alınır. Bütün ASCII kullanılacaksa offset gerekmez, karakter kodu doğrudan indis olur.
- Önce bütün tablo -1 yapılır. Sonra örüntü soldan sağa gezilir; her karakterin gözüne indisi yazılır. Aynı karakter birden çok geçiyorsa sonraki yazım öncekini ezer. Böylece en sağdaki indis kalır.

`NEEDLE` için tablo:

Karakter	N	E	D	L	Diğerleri
`rightSkip`	0	5	3	4	-1

```
void shiftLookUp(char pattern[], int rightSkip[], int offset, int tableN)
{
    int i, m = strlen(pattern);
    for (i = 0; i < tableN; i++)
        rightSkip[i] = -1;
    for (i = 0; i < m; i++)
        rightSkip[pattern[i] - offset] = i;
}
```

- Case 1 bu tabloyla kendiliğinden çözülür: örüntüde olmayan karakter için `skip = j - (-1) = j + 1`.

Boyer-Moore Kodu

```
int subStringSearch(char p[], char t[])   /* p: aranan, t: içinde arama yapılan */
{
    int i, j, skip = 0, N = strlen(t), M = strlen(p), stop = 0;
    i = 0;
```

```
    while ((i <= N - M) && (!stop)) {
        j = M - 1;
        while ((j >= 0) && (p[j] == t[i + j]))
            j--;
        if (j >= 0) {                       /* bulunamadı, kaydır */
            skip = j - rightSkip[t[i + j]];
            if (skip < 0)
                skip = 1;
            i += skip;
        }
        else                                /* j == -1: bulundu */
            stop = 1;
    }
    if (stop == 1)
        return i;
    else
        return -1;
}
```

Boyer-Moore Örneği

Hocanın tahtada çalıştığı örnek: örüntü `NEEDLE` ($M = 6$), metin `FINDINAHAYSTACKNEEDLEINA`.

Adım	i	Karşılaştırma	Uyumsuz karakter	skip	Yeni i
1	0	`t[5]` = `N` ile `E`	`N` (`rightSkip` = 0)	$5 - 0 = 5$	5
2	5	`t[10]` = `S` ile `E`	`S` (`rightSkip` = -1)	$5 - (-1) = 6$	11
3	11	`E`=`E` uyar; `t[15]` = `N` ile `L`	`N` (`rightSkip` = 0)	$4 - 0 = 4$	15
4	15	Bütün karakterler uyar	-	-	Bulundu, $i = 15$

Rabin-Karp Parmak İzi Yöntemi

Fikir: karakterleri tek tek karşılaştırmak yerine örüntüden bir sayı üretmek ve metindeki her M uzunluklu pencerenin sayısıyla karşılaştırmak.

- Örnek: yalnız büyük harfli metinde `A` = 0, `B` = 1, `C` = 2 alınırsa `BACA` = 1020 sayısıyla gösterilebilir.
- Uzun örüntülerde sayı çok büyür. Bu yüzden büyük bir asal Q ile mod alınır.

Slayttaki örnek: örüntü `26535`, $Q = 997$, örüntü özeti $26535 \bmod 997 = 613$. Metin `3141592653589793`.

i	Pencere	mod997
0	31415	508
1	14159	201
2	41592	715
3	15926	971
4	59265	442
5	92653	929
6	26535	613 (eşleşme, `return i = 6`)

- Özetler eşitse karakterler tek tek karşılaştırılarak doğrulanır. Çünkü farklı iki sayının modu aynı olabilir.

- Bir öğrenci, bölümlerin de saklanıp karşılaştırılabileceğini önerdi. Hoca bunun da çakışmayı ayırt edebileceğini kabul etti; ama yöntem bu adımda karakter karakter karşılaştırır.

Kayan Özet (Rolling Hash)

Her pencerenin sayısını baştan hesaplamak gereksizdir. B taban olmak üzere:

$$x_i = t_i \cdot B^{M-1} + t_{i+1} \cdot B^{M-2} + \cdots + t_{i+M-1} \cdot B^0$$

Sonraki pencerede baştaki rakam çıkar, sondaki rakam eklenir:

$$x_{i+1} = (x_i - t_i \cdot B^{M-1}) \cdot B + t_{i+M}$$

- Örnek ($B = 10$): $31415 \to (31415 - 3 \cdot 10^4) \cdot 10 + 9 = 14159$.
- Bir öğrenci baştaki rakamı atmak için bit kaydırma (shift) kullanılıp kullanılamayacağını sordu. Hoca, kaydırma operatörünün ikilik tabanda çalıştığını söyledi. Sağa bir kaydırma ikiye bölmektir ($8 \to 4$). Onluk tabanda kaydırma işe yaramaz; taban 2 olsaydı işe yarardı.

Karmaşıklık Karşılaştırması

Yöntem	Tipik durum	En kötü durum
Kaba kuvvet	yaklaşık N (gerçek metinlerde ~1,1N)	MN
Boyer-Moore	yaklaşık N/M	MN
Rabin-Karp	yaklaşık $7N$	$7N$ (Monte Carlo sürümü); doğrulamalı sürüm en kötü MN

- Boyer-Moore'un en kötü durumu: metin `ABBBBABBBB...`, örüntü `ABBBB` gibi. Sondan her karşılaştırma uyar, uyumsuzluk en başta çıkar. Neredeyse her konumda M karşılaştırma yapılır. Gerçek metinlerde bu çok nadirdir.

Ders sonrası notlar

- **Rabin-Karp kayan özet formülü.** Tahtadaki formül $x_{i+1} = x_i - t_i \cdot B^{M-1} + t_{i+M}$ şeklinde yazıldı. Baştaki rakam çıkarıldıktan sonra kalan kısım B ile çarpılmalıdır: $x_{i+1} = (x_i - t_i \cdot B^{M-1}) \cdot B + t_{i+M}$. Çarpım olmadan 31415'ten $1415 + 9 = 1424$ çıkar, 14159 çıkmaz. Mod alınırken çıkarma sonucu negatif olabileceği için Q eklenerek pozitife çekilir.
- **Rabin-Karp'ta kaydırma miktarı.** Hoca ilk anlatımda pencereyi "4 karakter atlayarak" ilerlettiğini söyledi. Yöntemde pencere her adımda bir karakter kayar. Hoca slayttaki örnekte bunu doğru gösterdi.
- $31415 \bmod 997$**.** Hoca 518 dedi. Doğrusu 508'dir ($997 \cdot 31 = 30907$, $31415 - 30907 = 508$); slaytta da 508 yazıyor. Hoca ikinci pencereyi de bir an "14179" diye okudu; doğrusu 14159'dur ve $14159 \bmod 997 = 201$'dir (tablodaki bütün değerler python ile denetlendi).
- **`NEEDLE` uzunluğu.** Hoca Boyer-Moore örneğine başlarken "m eşittir 5" dedi. `NEEDLE` 6 karakterdir, $M = 6$'dır; izleme tablosu da $j = M - 1 = 5$'ten başlar.
- **En sağdaki geçiş.** Hoca Case 2b'yi anlatırken bir yerde uyumsuz karakterin örüntüdeki "en soldaki" yerine göre kaydırıldığını söyledi. Doğrusu en sağdaki (rightmost) geçiştir; `rightSkip` tablosu da en sağdaki indisi tutar.
- **Case 1 formülü.** Tahtaya önce `i = i + j - 1` yazıldı. Hoca sonra `i = i + j + 1` diye düzeltti. Doğrusu `i + j + 1`'dir; `rightSkip` değeri -1 olduğu için `skip = j + 1` olur.
- **Kaba kuvvet en kötü durumu.** Hoca bir yerde $(N - 1) \cdot M$ dedi. Doğrusu $(N - M + 1) \cdot M$'dir.
- **Tipik durumda kaba kuvvet.** Hoca kaba kuvvetin "her zaman MN" olduğunu söyledi. MN en kötü durumdur. Gerçek metinlerde uyumsuzluk genelde ilk bir iki karakterde çıktığı için kaba kuvvet tipik olarak yaklaşık $1{,}1N$ karşılaştırma yapar. Kaba kuvvetin sorunu, en kötü durumun garanti altında olmamasıdır.

- **Rabin-Karp karmaşıklığı.** Hoca en kötü durum için “M çarpı M” dedi. Özet her eşitlikte doğrulanırsa en kötü durum MN’dir. Doğrulama yapılmayan (Monte Carlo) sürüm her zaman yaklaşık $7N$ işlem yapar ama çok küçük bir olasılıkla yanlış sonuç verebilir.
- **Horspool ve Boyer-Moore.** Uyumsuz karaktere göre kaydırma (bad character rule) Boyer ve Moore’un 1977 algoritmasının bir parçasıdır. Horspool 1980’de bu kuralın tek tablo kullanan basitleştirilmiş sürümünü önerdi. Derste anlatılan sürüm, Sedgewick’in kitabındaki basitleştirilmiş Boyer-Moore’dur.
- **`rightSkip` indisi.** Arama kodunda `rightSkip[t[i + j]]` yazıldı. Tablo `offset` ile kurulduğu için burada da `rightSkip[t[i + j] - offset]` kullanılmalıdır. Aksi halde `'A'` için 65. göze bakılır ve 26 boyutlu dizinin dışına çıkılır.
- **`rightSkip` başlangıç değeri.** Hoca “tablonun tamamını sıfırlıyorum” dedi ama -1 yazdı. Doğrusu -1’dir; 0 yazılırsa örüntüde olmayan karakterler 0. indiste varmış gibi görünür.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.